

\\$1

\\$1

EditPath in Kdenlive

Kdenlive, Melt/MLT, dataset samples, and installation

Technical report — 24 July 2026

This report explains how EditPath instruments Kdenlive editing sessions, how Kdenlive projects render through MLT and Melt, how raw commands become an editor-process replay and completed dataset sample, and how to install the stack on Linux, Windows, macOS, and containers.

1. Executive summary

EditPath is a capture, reconstruction, validation, and publication layer around Kdenlive. Kdenlive remains the interactive editor. EditPath records semantic commands, shortcuts, pointer gestures, project context, exact Kdenlive/MLT state sidecars, checkpoint proxies, asset hashes, and immutable evidence. Final media is rendered and validated independently. The editor replay is a separate video made from the raw event stream plus reconstructed before/after states; it is not a copy of the final movie.

2. Capture-to-sample architecture

Kdenlive + EditPath supervisor

- > raw-events-NNN.jsonl + exact .kdenlive.zst states
- > recovery/segment assembly and hash-chain validation
- > verified content-addressed assets
- > accepted-state reconstruction and Melt/MLT render
- > render_edit_process(): command/state editor replay
- > COMPLETED_SAMPLE/

3. What is recorded

- Session envelope: IDs, schema, timestamps, completion, and recovery boundaries.
- Project context: dimensions, FPS, aspect ratio, colorspace, audio, Kdenlive, and MLT versions.
- Raw UI evidence: ui.command, ui.shortcut, ui.gesture, transport actions, and labels/targets in sequence order.
-

Semantic edits: state.diff before/after hashes and changed clips, tracks, compositions, mixes, effects, and timeline duration.

-

Exact state: content-addressed compressed Kdenlive XML and checkpoint proxies.

-

Assets: stable IDs bound to copied files and SHA-256 hashes.

4. MLT and Melt rendering

MLT is the multimedia framework used by Kdenlive for producers, playlists, tractors, transitions, filters, and consumers. Melt is MLT's command-line multitrack/test editor. EditPath materializes the accepted Kdenlive state and asks Melt to render it with the recorded project profile.

Load the accepted project state and bind each clip to a verified asset.

Build an MLT profile from width, height, FPS, progressive mode, colorspace, and aspect-ratio fields in project.context.

Render through the avformat consumer, which uses FFmpeg libraries for encoding.

Probe with FFprobe and compare duration, dimensions, streams/audio, and visual similarity against the editor's reference render.

Quarantine failures caused by missing services, assets, state hashes, or media quality gates.

```
melt reconstructed.kdenlive \\  
-consumer avformat:reconstructed.mp4 \\  
vcodec=libx264 acodec=aac real_time=-1
```

The exact command depends on the encoder and project. Kdenlive projects may also require avformat, xml, qimage/Qt, title, frei0r, Lottie, and other MLT modules.

5. How the editor replay is made

The implementation is in edit_path/process_video.py.

build_replay_steps() selects the accepted branch baseline and state changes, then associates each step with a reconstructed semantic snapshot.

build_replay_moments() walks raw events in sequence order and joins gestures and shortcuts to nearby state diffs, creating before/after snapshots.

_render_scene() draws the editor-training view: project bin, project monitor, effects/properties, multitrack timeline, transport, pointer/keyboard feedback, command cards, selected clips, and semantic apply cards.

When exact source media exists, render_event() uses Melt/MLT for a state preview. If media or an MLT service is missing, the renderer records a warning and retains the semantic editor view.

Scenes are encoded frame-by-frame, concatenated, probed, and checked against the expected quantized duration. The replay report records moments, states,

accepted edits, event counts, preview quality, warnings, and output hash.

6. Completed sample layout

COMPLETED_SAMPLE/	
sample.json	authoritative entry point
output/final.mp4	editor reference video
edit-path/events.jsonl	canonical raw trajectory
edit-path/replay.mp4	editor-process replay
internal/final.kdenlive	portable reconstructed project
provenance/	original project/reference evidence
assets/	verified content-addressed media
metadata/	session, asset, and replay reports
evidence/	immutable raw segments
bundle-manifest.json	SHA-256/byte manifest

7. Installation

Linux: AppImage or Flatpak

Download Kdenlive from the official download page. For an AppImage, make it executable and launch it:

```
chmod +x Kdenlive-*.AppImage
```

The GUI bundle does not automatically guarantee a discoverable EditPath reconstruction runtime. Install/verify Melt/MLT, FFmpeg, FFprobe, Python, and Zstandard; install the complete MLT module set, including avformat, xml, and qimage/Qt services.

```
python3 -m edit_path doctor
melt --version
ffmpeg -version
ffprobe -version
```

Windows 10/11 x64

The repository documents a portable EditPath-Windows-x64.zip artifact. Extract it and launch EditPath.exe, not adjacent Kdenlive directly; the bundle carries instrumented Kdenlive, Melt/MLT, FFmpeg/FFprobe, Python, and reconstruction code. The engineering artifact is unsigned, so SmartScreen may warn. For a source setup, use PowerShell/Python 3, keep Melt and FFmpeg on PATH, and run `py -3 -m edit_path doctor`. MLT's Windows build uses Git, MinGW, and CMake.

macOS

Kdenlive supplies Intel and Apple Silicon downloads for supported macOS. The EditPath repository currently treats macOS as a native source-build path rather than a signed app bundle. Install Qt/KDE/MLT/FFmpeg dependencies, build EditPath, then run:

```
EDIT_PATH_REPO_ROOT="$PWD" ./build/bin/EditPath
python3 -m edit_path doctor
```

Docker Desktop can host reconstruction when a native runtime is inconvenient.

Containers, CI, and WSL

The pinned reconstruction/Containerfile installs Kdenlive/MLT modules, FFmpeg/AVFilter, audio backends, Python, and fonts.

```
docker build -t edit-path-reconstruction -f reconstruction/Containerfile .
```

```
docker run --rm --init -v "$PWD/session:/session:ro" \\  
-v "$PWD/dataset:/dataset" edit-path-reconstruction \\  
edit-path process /session /dataset
```

Use --init to reap child processes. On openSUSE/WSL, QtMultimedia QML imports are mandatory for Kdenlive's timeline; frei0r plugins and DejaVu or Liberation fonts are also called out for headless scenes.

8. Diagnostics and reproducibility

edit_path doctor must find Melt (or mlt-melt), FFmpeg, FFprobe, Zstandard, and a usable encoder. A runtime lock can pin Melt/MLT, FFmpeg, encoder, and container identity. Keep the runtime consistent across capture, reconstruction, and replay when visual comparison matters.

-

Hash mismatch: inspect recovery boundaries and the exact state chain.

-

Missing MLT service: install complete modules, not only the Melt executable.

-

Missing media: retain immutable assets and manifest hashes; otherwise exact previews may degrade while semantic replay remains available.

-

Qt/headless failure: configure the Qt platform/plugin path and install fonts.

9. Operator checklist

Launch the EditPath supervisor and edit in Kdenlive.

Close Kdenlive after the independent final render completes.

Run edit_path inspect and edit_path doctor.

Run edit_path process SESSION DATASET with the intended Melt/MLT runtime.

Review render and replay reports, then validate sample.json and the manifest.

Human-QA edit-path/replay.mp4: commands, shortcuts, gestures, semantic edits, and timeline states should be visible.

10. References

-

EditPath: <https://github.com/Parsewave-internal/edit-path>

-

Kdenlive downloads: <https://kdenlive.org/download/>

-

Kdenlive installation: https://docs.kdenlive.org/en/getting_started/installation.html

-

Melt documentation: <https://www.mltframework.org/docs/melt/>

-

MLT installation: <https://www.mltframework.org/docs/install/>

Check the linked official pages before production deployment because release versions and platform requirements change.